

Exercises: Introductory Course on Implicitly Switched ODEs and Filippov Sliding

July 18, 2025

Task 1

Consider this switched system on time horizon $T = [0, 30]$ with initial values $x_1(0) = 0, x_2(0) = 1000$ and parameter $p = (\frac{3}{4}, \frac{1}{4})$:

$$\begin{aligned}\frac{d}{dt}x_1(t) &= \begin{cases} q_1(t), & \text{if } x_1 \leq 750 \\ q_1(t) - q_2(t), & \text{if } x_1(t) > 750 \end{cases} \\ \frac{d}{dt}x_2(t) &= \begin{cases} 0, & \text{if } x_1 \leq 750 \\ -300, & \text{if } x_1(t) \in (750, 800) \\ 0 & \text{if } x_1(t) > 750 \end{cases}\end{aligned}$$

where

$$\begin{aligned}q_1(t) &= \frac{50}{27}t^2 - \frac{100}{3}t + \frac{400}{3} \\ q_2(t) &= (1000 - x_2)(p_1(t - 19) + p_2)\end{aligned}$$

- Solve this system using `ode45` and look at the solution. Does it look correct? Can you think of the reason why `ode45` produces the results it produces?
Define the right-hand side with the following function header: `function dx = task1RHS(x, t, p)`. Implement the helper functions `q1` and `q2` as local variables.
- Let's see how the solution responds to changes in the initial values: Plot the sensitivities of the solution using the provided `plotSensitivities_ode45(sol, @task1RHS, p, @ode45, options)`. Do they look correct?
- Now, solve this system using `IFDIFF`. What changed in comparison to Task 1 a)? Use the same function header and implementation of the helper functions as mentioned in a)
- Plot the sensitivities of the solution with `IFDIFF` using `plotSensitivities_ifdiff()`. How do the sensitivities differ from the sensitivities you previously computed in Task 1?
- Let's take a look at a different method: Use the `MaxStep` parameter to `odeset` to produce a forward solution: `options = odeset(..., 'MaxStep', 0.1);`. Does it look correct?
- Now, compute and plot the sensitivities of this solution. What do you notice?

For reference, your solution and sensitivity plots with `IFDIFF` should look like the following:

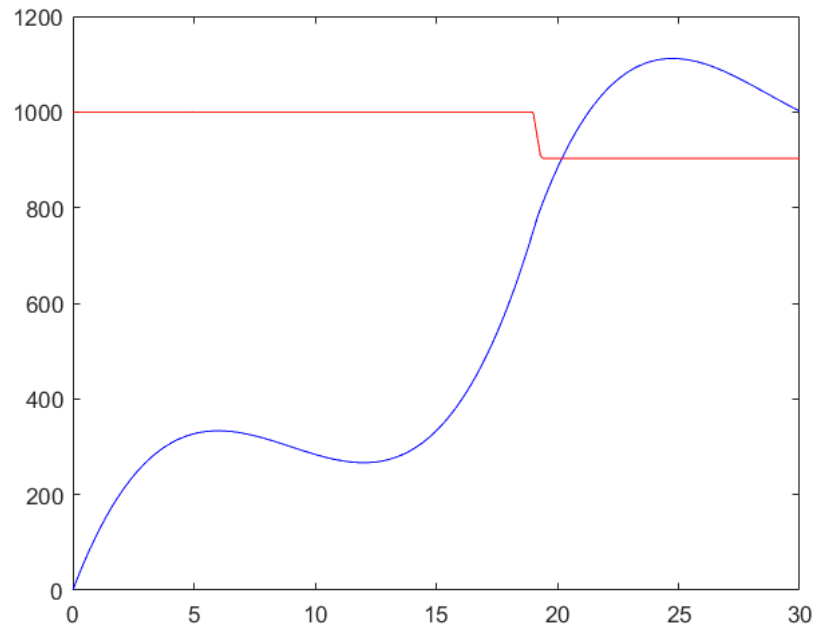


Figure 1: Solution plot for c)

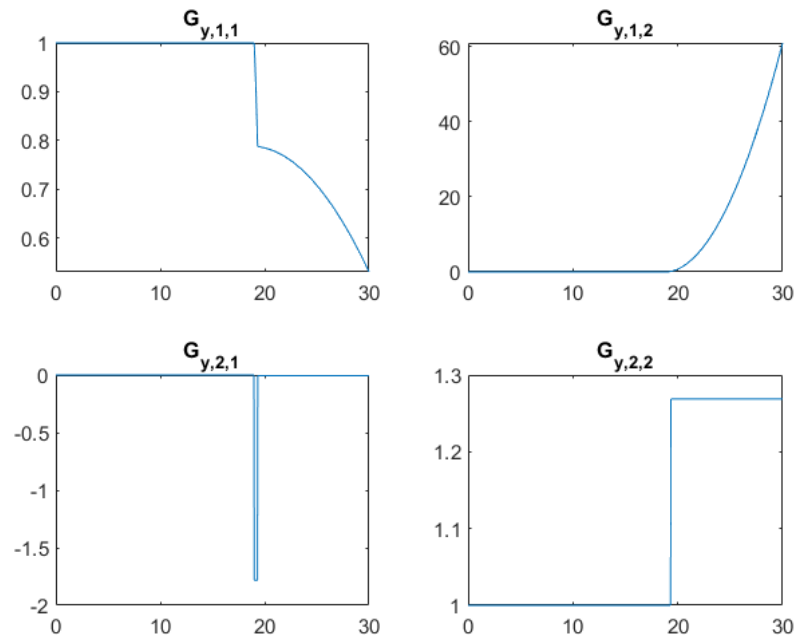


Figure 2: Sensitivities for d)

Task 2: State Jumps

Consider this discontinuous system

$$\begin{aligned}\frac{d}{dt}x_1(t) &= -\sin(t) \\ \frac{d}{dt}x_2(t) &= |x_1(t)|\end{aligned}$$

with state jumps

$$x_2(t) \leftarrow -x_2(t) \quad \text{whenever } x_1(t) \text{ crosses the threshold } p \text{ from below}$$

on the time horizon $T = [0, 40]$ with initial conditions $x_1(0) = 1$, $x_2(0) = 0$ and parameter $p = 0.5$.

- a) Solve this system using IFDIFF and plot the solution.

Hint: Create a file named `task2RHS.m` which defines the RHS function:

```
function dx = task2RHS(t, x, p)
```

Next, create a script named `solveTask2.m` where you setup the system parameters, call the appropriate IFDIFF functions to compute the solution and finally plot the solution.

- b) Compute and plot the sensitivities of the system w.r.t. initial values and parameters using IFDIFF with the VDE method. Can you explain intuitively why the sensitivities look the way that they do?

Hint: Extend your script `solveTask2.m` with a section that calls the appropriate IFDIFF functions to generate the sensitivity function. Use the `plotSensitivitiesSwitched` function to then plot the sensitivities.

Notation in the legend for the plot: $G_{y,i,j}/G_{p,i,j}$ is the sensitivity of the i -th solution component w.r.t. j -th initial value/parameter.

Try solving the system with different initial values and parameters to observe how the solution trajectory changes in response, to gain a rough understanding of the sensitivities.

For reference, your solution and sensitivity plots should look like the following:

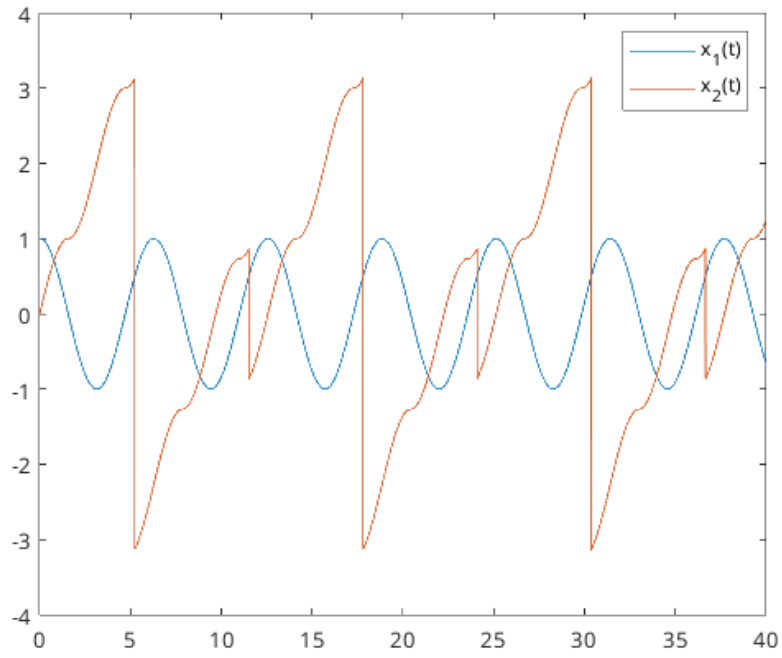


Figure 3: Task 2a: Solution

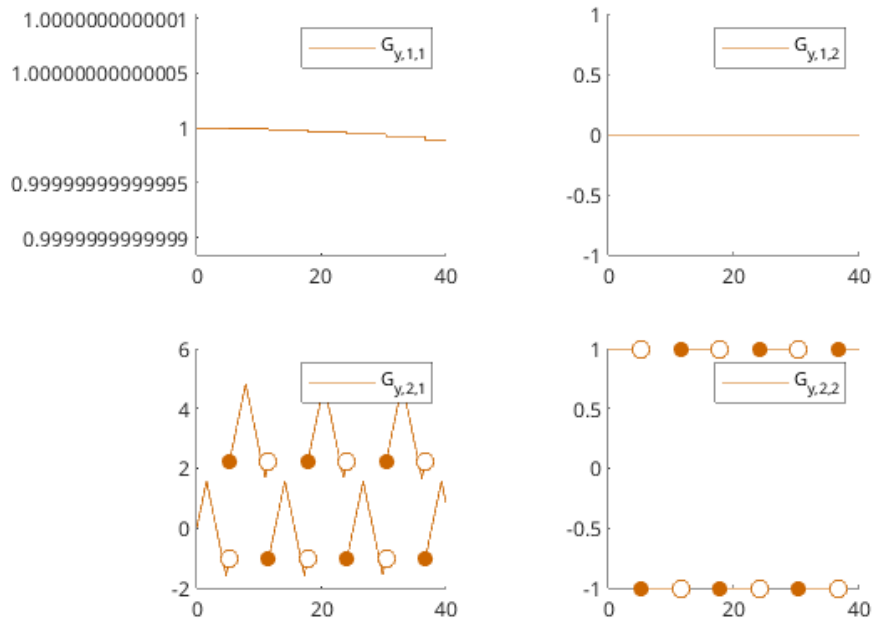


Figure 4: Task 2b: Sensitivities w.r.t. initial values

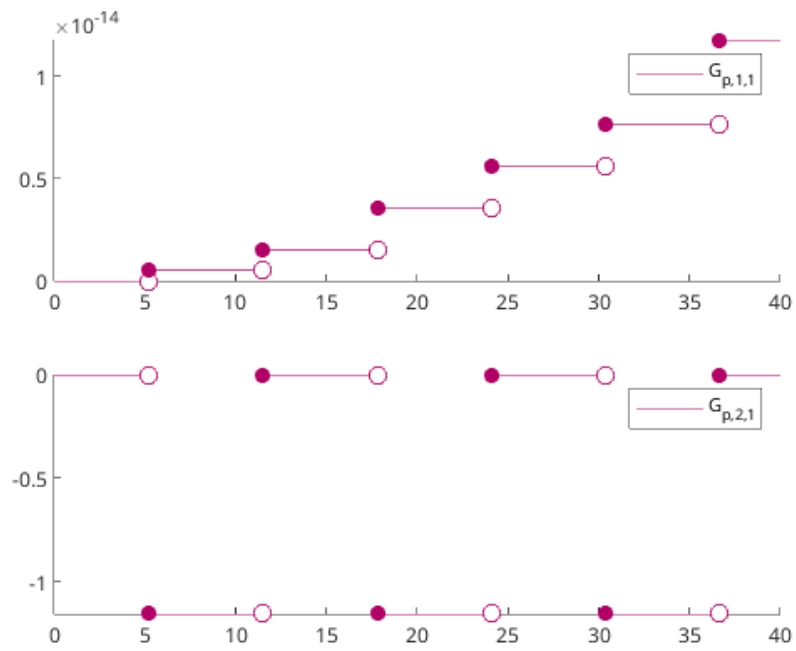


Figure 5: Task 2b: Sensitivities w.r.t. parameters

Task 3: Filippov Sliding

Filippov systems handle discontinuous right-hand sides using convex combinations. Consider the differential piecewise differential system on the time span $T = [0, 200]$:

$$\dot{x} = \begin{pmatrix} (r_1 - \beta_1 P)p_1 \\ r_2 p_2 \\ (eq_1 \beta_1 p_1 - m)P \end{pmatrix} \quad \text{if } H(p_1, p_2, P) > 0$$

$$\dot{x} = \begin{pmatrix} r_1 p_1 \\ (r_2 - \beta_2 P)p_2 \\ (eq_2 \beta_2 p_2 - m)P \end{pmatrix} \quad \text{if } H(p_1, p_2, P) < 0$$

where $\dot{x} = (\dot{p}_1, \dot{p}_2, \dot{P})^T$, $(p_1, p_2, P) \in \mathbb{R}_{\geq 0}^3$ and $H(p_1, p_2, P) = \beta_1 p_1 - a_q \beta_2 p_2$. The switching manifold of this system is the plane $S = H^{-1}(0)$.

Let us take the initial value $a = 0.28$ and these parameter values:

```
% fraction of prey in refuge:
m      = 0.845;
% growth rates of:
r1     = 1.124; % predator
r2     = 0.357; % prey
% conversion efficiency:
e      = 0.676;
% catchability coefficients:
q1     = 0.694;
q2     = 1.387;
% functional response parameters:
aq     = 0.634;
beta1  = 7.9;
beta2  = 1.925;
```

Solve this system using IFDIFF and compare it to the ode45 solution. Use the following function header: `function dx = task3RHS(t,x,p)` for the right hand side.

Hint: We provide the following plotter function you can use in your main script to plot the solutions. It can be found either as a `plotter.m` file in the HowTo folder on GitHub or you can copy it to your script from here:

```
function h = plotter(fignum, x, y, color, name, lw)
    figure(fignum); hold on;
    h = plot3(y(3,:), y(2,:), y(1,:), 'Color', color, 'LineWidth', lw, 'DisplayName', name);
    view([97 51]);
    grid on;
    box on;
    xlabel('Predator');
    ylabel('Prey 2');
    zlabel('Prey 1');
    legend('location', 'northeast');
    drawnow
    pause(1.0);
    set(fignum, 'Position', [200 250 750 375]);
end
```

For your reference, the solution plot should look like the following:

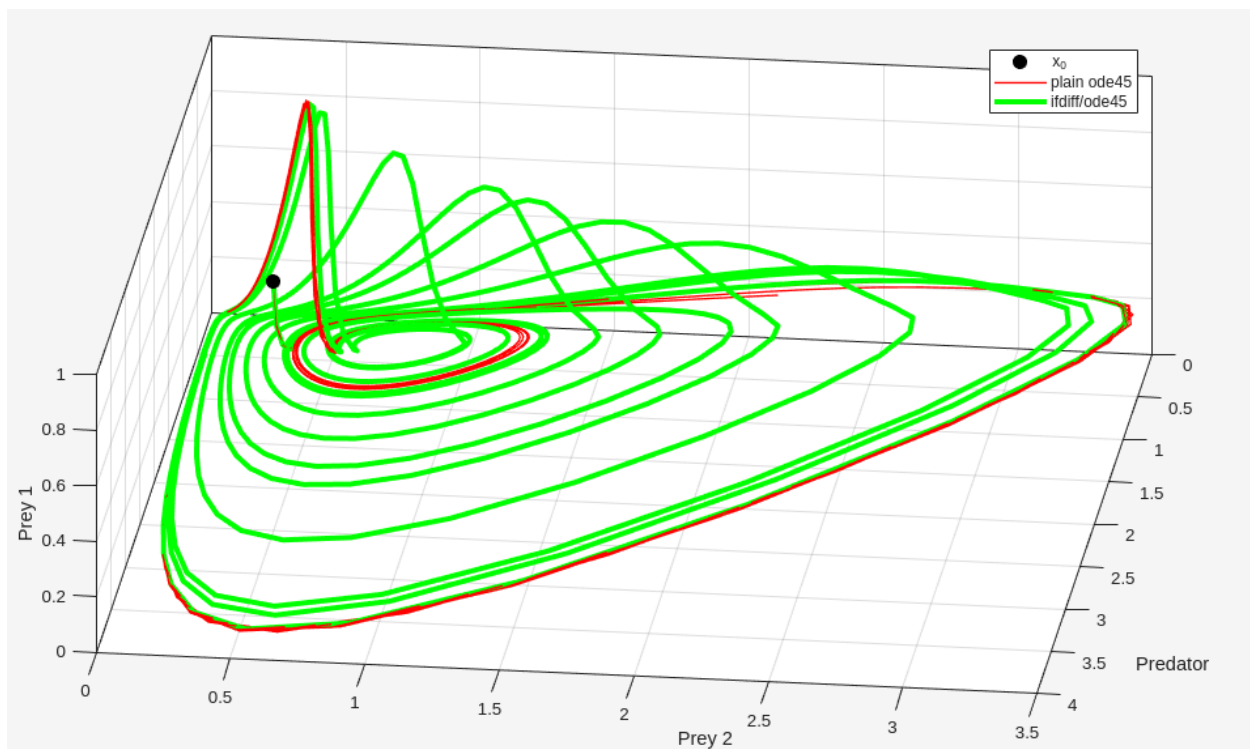


Figure 6: Solution plot to task 3. As you can see, the solution trajectory ode45 computes (red) differs from the IFDIFF solution (green). By solving the system via Explicit Euler, we can verify that IFDIFF is correct!

Task 4: Parameter Estimation

In this task, we finally try to put the sensitivity information to good use by solving a Parameter Estimation (PE) Problem. For simplicities sake, download the MATLAB files listed under **Exercises/Templates** from HEIBox and include them into your project.

The general structure of the task will consist of the following:

- Take an ODE Right-Hand-Side, choose initial values and parameters for the IVP (Initial Value Problem)
- Do one forward integration with these values and record the trajectory at a set of points in time t
- Disturbe the values at these times to create synthetic measurements
- Build a residual of any solution trajectory to the measurements and build the jacobian of this residual
- Choose new parameters and initial values and use `lsqnonlin` to try to get the original trajectory back

a) We want to do Parameter Estimation with a switched version of the Lotka Volterra system:

$$\dot{x} = \begin{cases} \frac{dx_1}{dt} = \alpha x_1 - \beta x_1 x_2, \\ \frac{dx_2}{dt} = \begin{cases} 5\gamma x_1 x_2 - 0.2\delta x_2, & \text{if } x_1 > 60, \\ \gamma x_1 x_2 - \delta x_2, & \text{if } x_1 \leq 60. \end{cases} \end{cases}, x_{\text{init}} = \begin{pmatrix} 20 \\ 10 \end{pmatrix}, \alpha = 0.2, \beta = 0.01, \gamma = 0.001, \delta = 0.1$$

Implement the dynamics above (in the file `LotkaVolterraRHS`) and perform one forward integration (with `ifdiff`) of this system on the time horizon $T = [0, 120]$.

Hint: You can get a vector of the four parameters from the function `getParams_LV()` which you downloaded with the rest of the template files.

b) Create synthetic measurements by randomly perturbing a set amount of values (e.g. at the time points $t = \text{tspan}(1):\text{tspan}(\text{end});$) along the initial trajectory.

Hint: `deval(sol, t)` returns all states at times t as a matrix object.

c) Implement the missing parts of `generateResFunction.m`. In particular, use your newly acquired knowledge of the sensitivity generation in `ifdiff` to build the jacobian matrix J of the residual vector r .

$$r(x_{\text{init}}, p) = \begin{bmatrix} \eta_0 - x(t_0, x_{\text{init}}, p) \\ \eta_1 - x(t_1, x_{\text{init}}, p) \\ \vdots \\ \eta_{118} - x(t_{118}, x_{\text{init}}, p) \end{bmatrix} = \begin{bmatrix} \eta_{01} - x_1(t_0, x_{\text{init}}, p) \\ \eta_{02} - x_2(t_0, x_{\text{init}}, p) \\ \eta_{11} - x_1(t_1, x_{\text{init}}, p) \\ \eta_{12} - x_2(t_1, x_{\text{init}}, p) \\ \vdots \\ \eta_{120,1} - x_1(t_{118}, x_{\text{init}}, p) \\ \eta_{120,2} - x_2(t_{118}, x_{\text{init}}, p) \end{bmatrix}$$

$$J(x_{\text{init}}, p) = \nabla r(x_{\text{init}}, p) = \begin{bmatrix} -\frac{\partial x(t_0)}{\partial x_{\text{init}}} & -\frac{\partial x(t_0)}{\partial p} \\ -\frac{\partial x(t_1)}{\partial x_{\text{init}}} & -\frac{\partial x(t_1)}{\partial p} \\ \vdots & \vdots \\ -\frac{\partial x(t_{118})}{\partial x_{\text{init}}} & -\frac{\partial x(t_{118})}{\partial p} \end{bmatrix}$$

Congratulations! You're done now. A resulting plot of the script might look similar to the following:

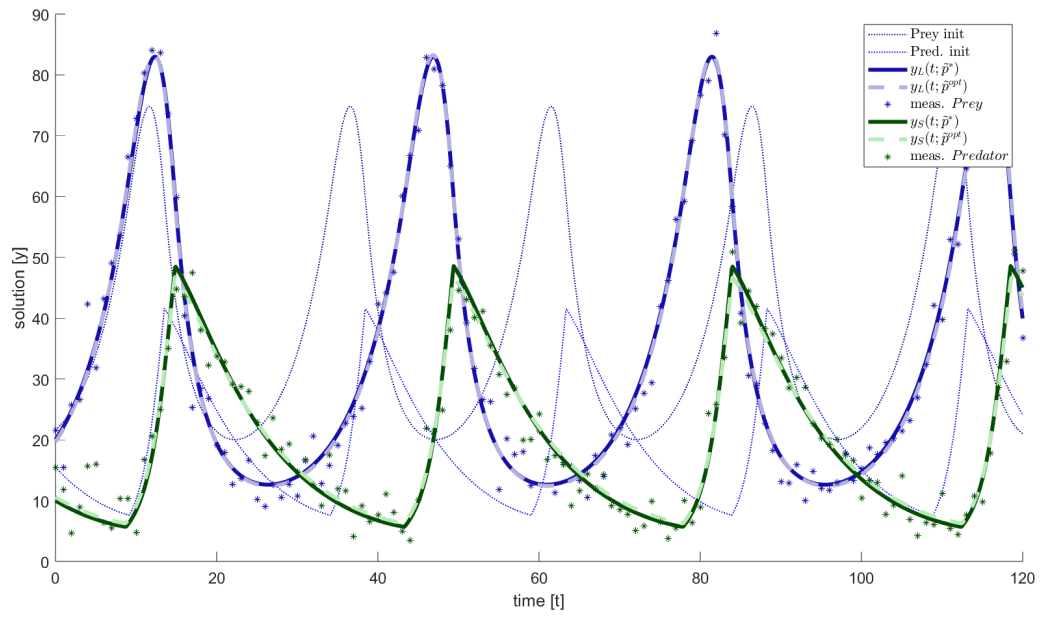


Figure 7: Trajectories from "real" solution, initial guess of PE, synthetic measurements and estimated solution

Appendix A: ODE Solving Cheat Sheet

Solving Predator-Prey System with ode45

$$\begin{aligned}\frac{dx_1}{dt} &= p_1x_1 - p_2x_1x_2 \quad (\text{Prey}) \\ \frac{dx_2}{dt} &= p_3x_1x_2 - p_4x_2 \quad (\text{Predator})\end{aligned}$$

with $T = [0, 500]$, $x_1(0) = 20$, $x_2(0) = 10$, and $p = (0.2, 0.01, 0.001, 0.1)$.

Implementation Steps:

1. **ODE Function:** Implement in a separate file `predatorpreyRHS.m`:

```
1 function dx = predatorpreyRHS(t, x, p)
2     dx = zeros(2,1);
3     dx(1) = p(1)*x(1) - p(2)*x(1)*x(2); % Prey dynamics
4     dx(2) = p(3)*x(1)*x(2) - p(4)*x(2); % Predator dynamics
5 end
```

2. **Setup Parameters and options:**

```
1 p = [0.2; 0.01; 0.001; 0.1];
2 initvals = [20; 10];
3 tspan = [0, 500];
4 options = odeset('RelTol', 1e-6, 'AbsTol', 1e-8) % set rel. and abs. tolerance
   appropriately
```

3. **Solve and Evaluate:**

```
1 % Modern solver syntax (returns solution structure)
2 integrator = @ode45;
3 sol = integrator(@(t,x) predatorpreyRHS(t,x,p), tspan, initvals, options);
4
5 % Evaluate
6 t_eval = linspace(tspan(1), tspan(2), 1000);
7 X_eval = deval(sol, t_eval);
8
9 % example evaluation at any desired point with deval
10 example_point = 20
11 x_example = deval(sol, example_point);
12 disp(x_example);
```

Warning: Avoid legacy syntax like `[T,X]=ode45(...)` because IFDIFF relies on the sol structure! A sol structure also enables sensitivity analysis and interpolation later on. To take a look at the sol structure, you can either find it in your Workspace in MATLAB or display it in the terminal using `disp`.

Plotting and Sensitivity Analysis

- **Basic Plotting:**

```
1 plot(t_eval, X_eval);  
2 legend('Prey', 'Predator');  
3 xlabel('Time'); ylabel('Population');
```

- **Sensitivity Analysis:** Use the provided helper function:

```
1 plotSensitivities_odexx(sol, @(t,x,p) predatorpreyRHS(t,x,p), p, integrator,  
    options);
```

Requires the original solver function handle and options.

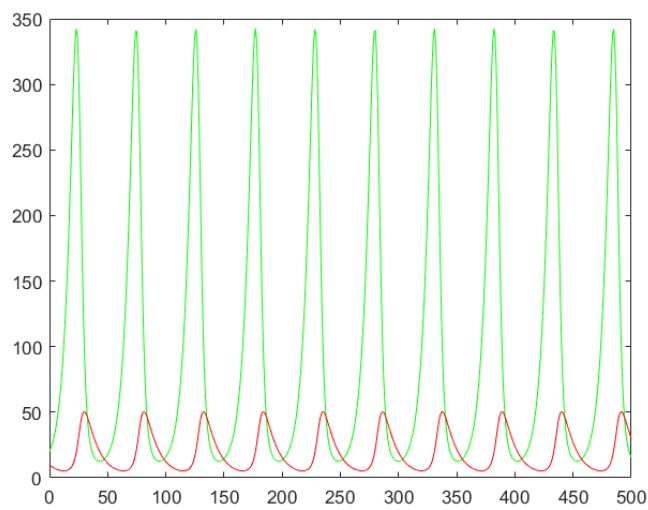


Figure 8: Solution trajectory of the Predator-Prey Example

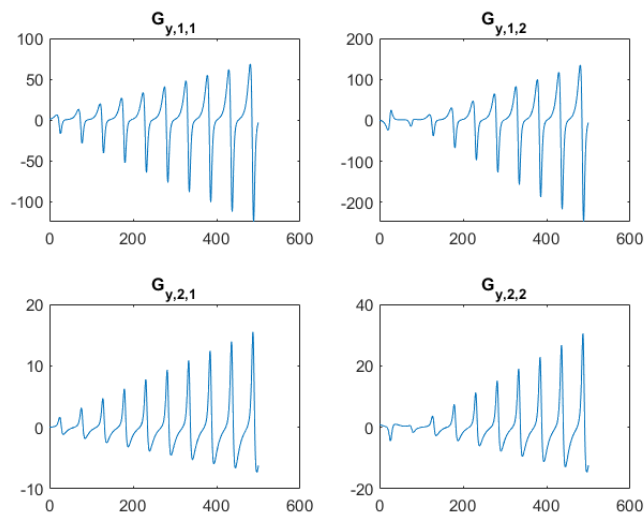


Figure 9: Sensitivities of the Predator-Prey Example

Appendix B: IFDIFF Cheat Sheet

RHS Function Requirements

- Signature: `function dx = myRHS(t, x, p)`. Keep in mind that the function file also needs to be named `myRHS.m`
- Helper functions work when defined either as:
 - in separate function files
 - as local variables
 - nested functionsbut **DO NOT** work as local functions of `myRHS.m` !
- **Warning:** Do not use `elseif` statements, instead use nested `if ... else ... end` statements to code the switching behavior.

IFDIFF Solution Setup: A Canonical Example

We consider the following ODE:

$$\dot{x} = f(t, x, p) = \begin{pmatrix} f_1(t, x, p) \\ f_2(t, x, p) \end{pmatrix}$$
$$f_1(t, x, p) = 0.01 \cdot t^2 + x_2^3 \quad f_2(t, x, p) = \begin{cases} 0 & \text{if } x_1 < p \\ 5 & \text{if } p \leq x_1 < p + 0.5 \\ 0 & \text{if } x_1 \geq p + 0.5 \end{cases}$$

with the initial value $x(0) = (0, 1)^T$ and parameter $p = 5.437$

1. Setup the the RHS and name the file `canonicalExampleRHS.m`

```
1 function dx = canonicalExampleRHS(t,x,p)
2
3 dx = zeros(2,1);
4 dx(1) = 0.01 * t.^2 + x(2).^3;
5 if x(1) < p(1)
6     dx(2) = 0;
7 else
8     if x(1) < p(1) + 0.5
9         dx(2) = 5;
10    else
11        dx(2) = 0;
12    end
13 end
14
15 end
```

2. **Main Script:** Setup the initial values, the time interval as well as parameter(s):

```
1 t0 = 0;
2 tf = 20;
3 timeinterval = [t0,tf];
4 initstates   = [1 0];
5 p            = 5.437;
```

3. Define options in the main script

```
1 integrator = @ode45;
2 odeoptions = odeset( 'AbsTol', 1e-10, 'RelTol', 1e-5);
3 filename = 'canonicalExampleRHS';
4 datahandle = prepareDatahandleForIntegration(filename, 'integrator', func2str(
    integrator), 'options', odeoptions);
```

4. Create a solution structure and evaluate

```
1 T = t0:0.1:tf;
2 sol_ifdiff = solveODE(datahandle, timeinterval, initstates, p);
3 Y_ifdiff = deval(sol_ifdiff, T);
```

5. Plot the solution

```
1 plot(T, Y_ifdiff);
2 xlabel('Time (s)');
3 ylabel('State Variables');
4 title('IFDIFF Solution');
5 grid on;
```

6. Compute and plot the sensitivities. There are two methods to do so. For the first one, you need to use `generateFDstep` so get finite differences and then use `generateSensitivityFunction` (which uses the VDE method in this example) to compute the sensitivities. Afterward, you can access and plot the sensitivities by appropriately reshaping the matrices in the `sensitivities` struct array, which are stored in the fields `Gy` for sensitivities w.r.t. initial values and `Gp` for sensitivities w.r.t. parameters respectively.

```
1 %% Step 5: Compute sensitivities manually
2
3 dim_y = size(Y_ifdiff, 1);
4 dim_p = length(p); % dim_p = 1
5
6 FDstep = generateFDstep(dim_y, dim_p, 'hy_rel_flag', true, 'hp_rel_flag', true,
    'hy_min', 1e-6, 'hp_min', 1e-6);
7 sensFun = generateSensitivityFunction(datahandle, sol_ifdiff, FDstep, 'method',
    'VDE', 'integrator_options', options);
8
9 sensitivities = sensFun(T); % Compute sensitivities at time points in T
```

Alternatively, you may use the provided `plotSensitivities_ifdiff` function which already computes the sensitivities internally and just outputs a plot:

```
1 plotSensitivities_ifdiff(datahandle, sol_ifdiff, p);
```

Your plots should look like the following:

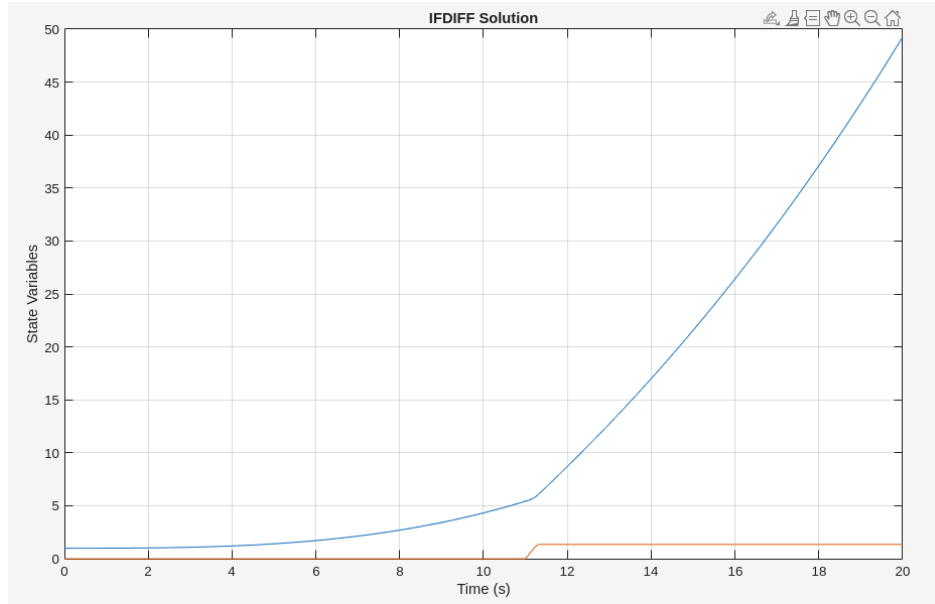


Figure 10: Canonical Example solution with IFDIFF

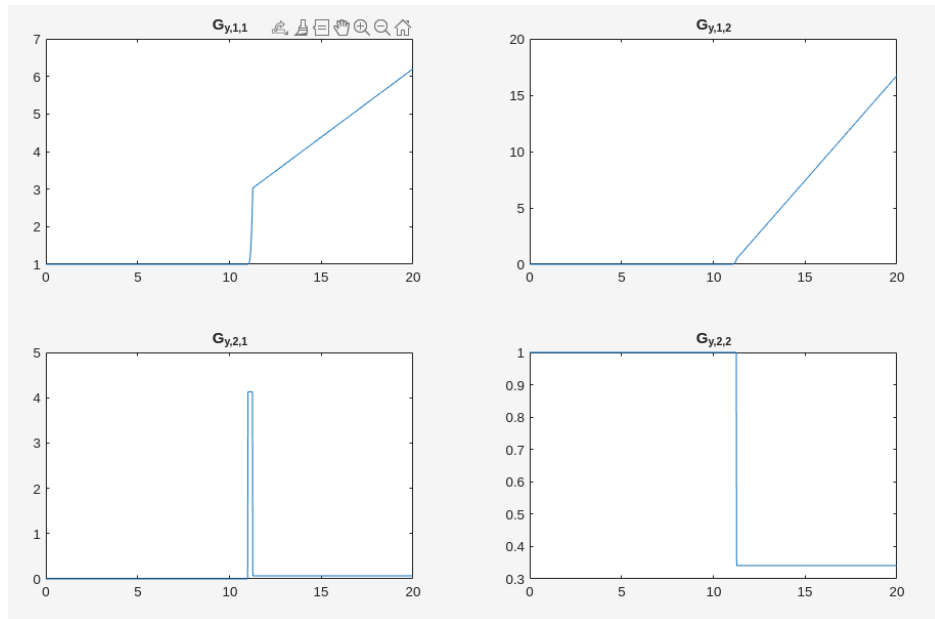


Figure 11: Sensitivities for the Canonical Example with IFDIFF

Specifying state jumps in a RHS

A state jump may be specified at any point in the RHS via a special if-block:

```
1 function dx = myRHS(t, x, p)
2     ...
3     if ifdiff_jumpif(jumpCondition, jumpDirection)
4         ... <statements computing the jumpIncrement> ...
5         ifdiff_update(jumpIncrement)
6     end
7     ...
8 end
```

When the first argument of the `ifdiff_jumpif` function (`jumpCondition`) crosses zero in the direction specified by the second argument (`jumpDirection`), i.e. negative to positive (1), positive to negative (-1) or both (0), IFDIFF will increment the state vector in the solution by the first argument of the `ifdiff_update` function (`jumpIncrement`).

Warning: The state vector and `jumpIncrement` must have matching dimensions. In particular, the `jumpIncrement` must be a column-vector with the same number of elements as the state vector.

Warning: IFDIFF will never execute the body of the if-block directly in the RHS, so make sure that the statements in the if-block are only used to compute the `jumpIncrement`. In particular, any variables exclusively defined in the if-block must not be used outside of the if-block.

Plotting state jump sensitivities

Solve the system, compute the sensitivities and then call the `plotSensitivitiesSwitched` function. Here is an example:

```
1 % RHS params
2 tspan = [0, 10];
3 h0 = 0;
4 v0 = 10;
5 x0 = [h0; v0];
6 p = 9.8;
7 % Solver params
8 solver = @ode45;
9 options = odeset('AbsTol', 1e-8, 'RelTol', 1e-6);
10 % Call IFDIFF
11 dh = prepareDatahandleForIntegration(@bounceRHS, 'integrator', solver, 'options',
    options);
12 sol = solveODE(dh, tspan, x0, p);
13
14 dim_y = size(sol.y, 1);
15 dim_p = length(p);
16 FDstep = generateFDstep(dim_y, dim_p);
17 sensFun = generateSensitivityFunction(dh, sol, FDstep, 'method', 'VDE');
18
19 plotSensitivitiesSwitched(sol, sensFun, tspan);
```